

Ampliación de Bases de Datos

Grado en Ingeniería del Software

Curso académico 2026/2027

TPI 2

Arquitectura Big Data y Procesamiento Analítico con Spark SQL Integrado en Sistemas de Información Empresarial

Estudiante: Alfonso Nicolás Martínez Márquez
Profesor: Dr. Anier Soria Lorente
Fecha de entrega: Octubre de 2026
Repositorio GitHub: https://github.com/alfonsomartinez04/Movilidad_Loyola

Índice

1. Problema real y contexto **2**

2. Necesidad relacionada con los datos y objetivos **2**

2.1. Necesidad identificada 3

2.2. Objetivos del trabajo 3

3. Conceptos y tecnologías aplicadas **3**

3.1. Apache Spark y Computación en Memoria (In-Memory) 3

3.2. Spark SQL, DataFrames y el Optimizador Catalyst (Apartado 5.4) 4

3.3. Integración Empresarial mediante JDBC (Apartado 5.5) 4

4. Desarrollo del trabajo **4**

4.1. Despliegue del entorno con Docker Compose 4

4.2. Dataset masivo de movilidad y esquema inferido 5

4.3. Consultas Analíticas en Spark SQL 5

4.3.1. Consulta 1: Rendimiento y Emisiones por Línea de Transporte . . . 6

4.3.2. Consulta 2: Detección de Franjas y Horas Punta 6

4.3.3. Consulta 3: Ahorro de Emisiones y Distritos Residenciales 7

4.4. Canal de Integración Empresarial mediante JDBC 7

5. Resultados obtenidos **8**

6. Análisis e interpretación de los resultados **9**

7. Decisiones tomadas y justificación **10**

8. Conclusiones y trabajo futuro **10**

9. Declaración de uso de inteligencia artificial **11**

10. Referencias **11**

1. Problema real y contexto

El presente trabajo aborda una problemática crítica en la gestión y planificación operativa de grandes infraestructuras universitarias: **la movilidad diaria y el transporte multimodal de la comunidad universitaria** hacia el Campus de la Universidad Loyola en Sevilla (Dos Hermanas).

En el trabajo anterior (TPI 1), analizamos espacialmente el alojamiento estudiantil en las zonas metropolitanas cercanas al campus (Entrenúcleos, Montequinto, Bellavista, Dos Hermanas y Sevilla Centro). Este TPI 2 da el salto lógico de ingeniería: responder a **cómo se desplazan diariamente esos miles de estudiantes y trabajadores hacia el campus**.

Cada día, la llegada simultánea de miles de estudiantes genera una masa crítica de información registrada por múltiples fuentes heterogéneas:

- Tornos y canceladoras del Metro de Sevilla (Línea 1 en Olivar de Quintos y Europa).
- Sensores de paso de los autobuses lanzadera de Loyola y de la línea M-133 del Consorcio de Transportes Metropolitano de Sevilla.
- Lecturas de las líneas urbanas de Dos Hermanas (Línea 5).
- Aforadores de la red ciclista metropolitana en Entrenúcleos.
- Barreras de control de acceso a los aparcamientos del campus (vehículo privado y carpooling).

A lo largo de un único cuatrimestre académico, estas fuentes generan cientos de miles de registros de desplazamientos individuales. La Universidad Loyola necesita analizar este volumen masivo de datos para resolver cuellos de botella en horas punta (07:30–09:30 h), planificar refuerzos de transporte público y cuantificar el ahorro real de huella de carbono (CO_2).

El procesamiento directo de millones de eventos en una base de datos relacional tradicional transaccional colapsaría los tiempos de respuesta del sistema corporativo. Por ello, se diseña una arquitectura **Big Data** desacoplada basada en **Apache Spark** y **Spark SQL**, que transforma y sintetiza la analítica masiva en memoria y vuelca los indicadores consolidados (KPIs) en una base de datos relacional empresarial **PostgreSQL** mediante **JDBC**.

Todo el código fuente, scripts de PySpark, dataset de 100.000 filas, cuadro de mando interactivo y entorno Docker Compose están estructurados y publicados en el repositorio del proyecto: `https://github.com/alfonsomartinez04/Movilidad_Loyola`.

2. Necesidad relacionada con los datos y objetivos

2.1. Necesidad identificada

Para dar respuesta al vicerrectorado de infraestructuras y sostenibilidad se requieren abordar las tres dimensiones clásicas del Big Data ($3V$):

1. **Volumen:** Procesar una muestra de 100.000 eventos de desplazamiento por cuatrimestre, con métricas de duración, distancia, retrasos, ocupación y emisiones asociadas.
2. **Velocidad:** Capacidad de computar agregaciones temporales y rankings analíticos en segundos, sin paralizar la operativa universitaria.
3. **Variedad:** Integrar registros con datos temporales, geográficos (distritos de origen), operativos (líneas de autobús y metro) y ecológicos (kg de CO_2).

2.2. Objetivos del trabajo

De acuerdo con las directrices del Tema 5 (*Big Data y bases de datos a gran escala*), los objetivos técnicos son:

- **Despliegue distribuido (Contenedores):** Orquestar un clúster de Apache Spark 3.5.1 junto con un servidor empresarial PostgreSQL mediante Docker Compose.
- **Procesamiento analítico masivo con Spark SQL (Apartado 5.4):** Implementar consultas analíticas sobre DataFrames en memoria para calcular horas punta, reparto modal, retrasos medios y emisiones de CO_2.
- **Integración con Sistemas de Información Empresarial (Apartado 5.5):** Establecer una tubería de exportación mediante el conector JDBC de PostgreSQL para alimentar las tablas del sistema corporativo.
- **Cuadro de Mando de Explotación:** Desarrollar un visor web interactivo en HTML5 y Chart.js para que los gestores universitarios tomen decisiones basadas en los KPIs generados.

3. Conceptos y tecnologías aplicadas

3.1. Apache Spark y Computación en Memoria (In-Memory)

Frente al paradigma clásico de Hadoop MapReduce (que escribía en disco duro en cada etapa intermedia), Apache Spark realiza todas las transformaciones directamente en la

memoria RAM. Esto reduce drásticamente el coste de entrada/salida (I/O), permitiendo acelerar las consultas analíticas hasta 100 veces.

3.2. Spark SQL, DataFrames y el Optimizador Catalyst (Apartado 5.4)

Spark SQL es el módulo especializado de Apache Spark para el procesamiento estructurado. Permite interactuar con los datos a través de dos mecanismos complementarios:

- **DataFrames:** Colecciones distribuidas organizadas en columnas tipadas, con evaluación perezosa (*lazy evaluation*).
- **Sintaxis SQL Estándar:** Permite registrar vistas temporales mediante `createOrReplaceTempView` y lanzar consultas SQL estándar ANSI (`SELECT`, `GROUP BY`, `HAVING`, `CASE`, *Window Functions*).
- **Optimizador Catalyst:** Motor interno que analiza el árbol sintáctico de la consulta y aplica optimizaciones lógicas (poda de columnas, reordenación de predicados y resolución de expresiones) antes de compilar el plan de ejecución físico en código JVM.

3.3. Integración Empresarial mediante JDBC (Apartado 5.5)

En los ecosistemas reales de ingeniería del software, Spark no sustituye a la base de datos empresarial, sino que actúa como **motor analítico ETL** (*Extract, Transform, Load*):

1. Ingiere grandes volúmenes de datos en bruto (CSV, Parquet, JSON).
2. Ejecuta transformaciones pesadas y síntesis de métricas en memoria.
3. Escribe las tablas de resultados compactas en PostgreSQL mediante el protocolo de red **JDBC** (*Java Database Connectivity*), utilizando el controlador `postgresql-42.7.3.jar`.
4. El sistema de información corporativo o cuadro de mando consulta PostgreSQL sin sufrir degradación.

4. Desarrollo del trabajo

4.1. Despliegue del entorno con Docker Compose

Se diseñó un entorno reproducibilidad total en el archivo `compose.yaml`, definiendo dos servicios en una red privada tipo *bridge* (`loyola-net`):

- **`loyola-spark-master`:** Contenedor con la imagen oficial `apache/spark:3.5.1-python3`, configurado con el entorno PySpark, Java 11 y la Web UI en el puerto 8080.
- **`loyola-postgres-empresa`:** Contenedor con PostgreSQL 16 configurado en el puerto local 5433 y base de datos `universidad_empresa_db`.

```
PS C:\WINDOWS\system32> docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED          STATUS          PORTS
                                                                  NAMES
fdc59df3000b   apache/spark:3.5.1-python3   "/opt/entrypoint.sh …"   20 minutes ago   Up 20 minutes   0.0.0.0:7077->7077
/tcp, [::]:7077->7077/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   loyola-spark-master
8c5a37018e83   postgis/postgis:16-3.4    "docker-entrypoint.s…"   20 minutes ago   Up 20 minutes   0.0.0.0:5433->5432
/tcp, [::]:5433->5432/tcp                                         loyola-postgres-empresa
```

Figura 1: Verificación de los contenedores Docker en PowerShell: `loyola-spark-master` (puertos 8080 y 7077) y `loyola-postgres-empresa` (puerto 5433).

4.2. Dataset masivo de movilidad y esquema inferido

Creamos el script generador `generar_dataset_movilidad.py` que simula 100.000 desplazamientos hacia el Campus de Loyola durante el cuatrimestre académico. El archivo `datos_movilidad_loyola_100k.csv` (23,6 MB) incluye 17 campos representativos: marcas temporales, día de la semana, franja horaria, distrito de origen, destino en el campus, línea de transporte, distancia, duración, retraso, ocupación, CO_2 emitido, CO_2 evitado y satisfacción del usuario.

Al cargar el archivo CSV con la opción `inferSchema=true`, el motor Catalyst de Apache Spark escanea la cabecera e infiere automáticamente los tipos de datos en un esquema estructurado (Figura 2):

```
--- ESQUEMA INFERIDO POR SPARK (CATALYST ENGINE) ---
root
 |-- id_viaje: integer (nullable = true)
 |-- timestamp: timestamp (nullable = true)
 |-- fecha: date (nullable = true)
 |-- hora: integer (nullable = true)
 |-- dia_semana: string (nullable = true)
 |-- franja_horaria: string (nullable = true)
 |-- distrito_origen: string (nullable = true)
 |-- destino: string (nullable = true)
 |-- medio_transporte: string (nullable = true)
 |-- linea_servicio: string (nullable = true)
 |-- distancia_km: double (nullable = true)
 |-- duracion_min: double (nullable = true)
 |-- retraso_min: double (nullable = true)
 |-- ocupacion_unidad_pct: double (nullable = true)
 |-- co2_emitido_kg: double (nullable = true)
 |-- co2_evitado_kg: double (nullable = true)
 |-- satisfaccion_usuario: integer (nullable = true)
```

Figura 2: Esquema relacional inferido en memoria por el motor Catalyst de Apache Spark sobre el dataset masivo de 100.000 filas.

4.3. Consultas Analíticas en Spark SQL

El script `procesar_spark_loyola.py` implementa las siguientes consultas estructuradas:

```
spark = SparkSession.builder \
    .appName("AnaliticaMovilidadCampusLoyola") \
    .config("spark.driver.memory", "2g") \
    .config("spark.jars", "/opt/spark/work-dir/postgresql-42.7.3.jar") \
    .getOrCreate()

df_raw = spark.read.option("header", "true") \
                   .option("inferSchema", "true") \
                   .csv("/opt/spark/work-dir/datos_movilidad_loyola_100k.csv")

df_raw.createOrReplaceTempView("movilidad_campus")
```

Listing 1: Inicialización de SparkSession y registro de la vista temporal

4.3.1. Consulta 1: Rendimiento y Emisiones por Línea de Transporte

Permite calcular el volumen total de viajeros, tiempo medio de desplazamiento, retraso medio y CO_2 acumulado:

```
SELECT
    linea_servicio AS linea,
    medio_transporte,
    COUNT(*) AS total_viajes,
    ROUND(AVG(duracion_min), 2) AS duracion_media_min,
    ROUND(AVG(retraso_min), 2) AS retraso_medio_min,
    ROUND(AVG(ocupacion_unidad_pct), 2) AS ocupacion_media_pct,
    ROUND(SUM(co2_emitido_kg), 2) AS co2_total_kg
FROM movilidad_campus
GROUP BY linea_servicio, medio_transporte
ORDER BY total_viajes DESC;
```

Listing 2: Consulta Spark SQL por línea de servicio

4.3.2. Consulta 2: Detección de Franjas y Horas Punta

Clasifica la congestión del tráfico universitario mediante sentencias condicionales `CASE WHEN`:

```
SELECT
    franja_horaria AS franja,
    MIN(hora) AS hora_inicio,
    MAX(hora) AS hora_fin,
```

```
    COUNT(*) AS total_desplazamientos,
    ROUND((COUNT(*) * 100.0) / 100000.0, 2) AS pct_sobre_total,
    FIRST(medio_transporte) AS medio_principal,
    CASE
        WHEN COUNT(*) >= 35000 THEN 'Critico (Punta)'
        WHEN COUNT(*) >= 20000 THEN 'Moderado'
        ELSE 'Fluido (Valle)'
    END AS nivel_congestion
FROM movilidad_campus
GROUP BY franja_horaria
ORDER BY total_desplazamientos DESC;
```

Listing 3: Detección de Horas Punta en Spark SQL

4.3.3. Consulta 3: Ahorro de Emisiones y Distritos Residenciales

Calcula la huella ecológica evitada respecto al vehículo privado en función del origen del estudiante:

```
SELECT
    distrito_origen,
    COUNT(*) AS total_viajeros,
    FIRST(medio_transporte) AS medio_mas_usado,
    ROUND(AVG(duracion_min), 2) AS tiempo_promedio_min,
    ROUND(AVG(distancia_km), 2) AS distancia_media_km,
    ROUND(SUM(co2_evitado_kg), 2) AS co2_evitado_kg
FROM movilidad_campus
GROUP BY distrito_origen
ORDER BY total_viajeros DESC;
```

Listing 4: Ahorro ecológico por distrito en Spark SQL

4.4. Canal de Integración Empresarial mediante JDBC

Una vez computadas las consultas en memoria, Spark SQL escribe directamente en las tablas relacionales de PostgreSQL:

```
jdbc_url = "jdbc:postgresql://postgres-empresa:5432/
   universidad_empresa_db"
props = {
    "user": "loyola_admin",
    "password": "loyola_password",
    "driver": "org.postgresql.Driver"
}

```

```
8  df_lineas.write.jdbc(url=jdbc_url, table="kpis_lineas_transporte"
       , mode="overwrite", properties=props)
9  df_franjas.write.jdbc(url=jdbc_url, table="kpis_franjas_horarias"
       , mode="overwrite", properties=props)
10 df_origenes.write.jdbc(url=jdbc_url, table="
       kpis_origenes_estudiantes", mode="overwrite", properties=props)
```

Listing 5: Volcado de DataFrames a PostgreSQL mediante JDBC

5. Resultados obtenidos

La ejecución del pipeline analítico con Apache Spark sobre los 100.000 registros concluyó con éxito en la consola de PowerShell en tan solo **12,977 segundos**. En la Figura 3 se aprecia la ejecución real en la terminal mostrando los resultados de la Consulta Spark SQL 1 sobre las líneas de transporte.

```
--------------------------------------------------------------------------------
 CONSULTA SPARK SQL 1: Analisis de Rendimiento y CO2 por Linea de Transporte
--------------------------------------------------------------------------------
+-----------------------------------------+-----------------------------+------------+-----------------+-----------------+------------------+-------
-----+
|linea                                    |medio_transporte             |total_viajes|duracion_media_min|retraso_medio_min|ocupacion_media_pct|co2_tot
al_kg|
+-----------------------------------------+-----------------------------+------------+-----------------+-----------------+------------------+-------
-----+
|Metro L1 (Olivar Quintos) + Lanzadera Loyola|Metro L1 + Bus Lanzadera     |34993       |18.82            |2.11             |75.45             |5311.23
     |
|Linea M-133 Sevilla (Prado) - Loyola     |Autobus Metropolitano Consorcio |21949       |27.93            |6.82             |75.69             |5296.22
     |
|Red Ciclista Metropolitana Entrenúcleos  |Bicicleta / Patinete Electrico  |16184       |30.49            |1.5              |75.46             |0.0
     |
|Linea 5 Urbano DH (Plaza Arenal - Loyola)|Autobus Urbano Dos Hermanas     |11880       |31.24            |6.82             |75.64             |3216.0
     |
|Carpooling App Estudiantes Loyola        |Vehiculo Compartido (Carpooling)|8930        |11.91            |1.5              |75.64             |2721.28
     |
|Acceso Vial Autovía A-4 / SE-40          |Vehiculo Privado Individual     |6064        |11.21            |1.49             |75.99             |5876.92
     |
+-----------------------------------------+-----------------------------+------------+-----------------+-----------------+------------------+-------
-----+
```

Figura 3: Ejecución real en terminal de la Consulta Spark SQL 1: rendimiento, ocupación, retrasos y emisiones acumuladas por línea de servicio.

Asimismo, la Consulta Spark SQL 2 clasificó los desplazamientos en franjas temporales y categorizó el nivel de saturación mediante cláusulas `CASE WHEN` (Figura 4):

```
--------------------------------------------------------------------------------
 CONSULTA SPARK SQL 2: Deteccion de Horas Punta y Nivel de Congestion
--------------------------------------------------------------------------------
+--------------------------+-----------+--------+--------------------+---------------+-----------------------------+----------------+
|franja                    |hora_inicio|hora_fin|total_desplazamientos|pct_sobre_total|medio_principal              |nivel_congestion|
+--------------------------+-----------+--------+--------------------+---------------+-----------------------------+----------------+
|Punta Mañana (07:30-09:30)  |7          |9       |44879               |44.88          |Metro L1 + Bus Lanzadera     |Critico (Punta) |
|Punta Mediodía (13:30-15:30)|13         |15      |30049               |30.05          |Metro L1 + Bus Lanzadera     |Moderado        |
|Valle Mañana (10:00-13:00)  |10         |12      |15016               |15.02          |Vehiculo Privado Individual  |Fluido (Valle)  |
|Tarde/Noche (16:00-21:00)   |16         |21      |10056               |10.06          |Bicicleta / Patinete Electrico|Fluido (Valle)  |
+--------------------------+-----------+--------+--------------------+---------------+-----------------------------+----------------+
```

Figura 4: Ejecución real en terminal de la Consulta Spark SQL 2: clasificación por franjas horarias y detección de horas punta críticas.

En el Cuadro 1 se resumen los KPIs agregados obtenidos para cada línea de transporte:

Tabla 1: KPIs analíticos por línea de transporte calculados por Spark SQL.

Línea de Servicio	Medio	Viajes	Duración (m)	Retraso (m)	CO_2 Total
Metro L1 + Bus Lanzadera	Metro + Bus	34.993	18,82 min	2,11 min	5.311 kg
Línea M-133 (Consorcio)	Autobús Metrop.	21.949	27,93 min	6,82 min	5.296 kg
Red Ciclista Entrenúcleos	Bici / Patinete	16.184	30,49 min	1,50 min	0,00 kg
Línea 5 Urbano DH	Autobús Urbano	11.880	31,24 min	6,82 min	3.216 kg
Carpooling Estudiantes	Coche Compart.	8.930	11,91 min	1,50 min	2.721 kg
Vehículo Privado A-4	Coche Individual	6.064	11,21 min	1,49 min	5.877 kg

Para comprobar que la integración con el sistema de información empresarial ha funcionado correctamente, en la Figura 5 se observa la salida de la consola durante la fase de volcado JDBC. El motor Spark conecta con `universidad_empresa_db` en PostgreSQL y vuelca las tres tablas de KPIs analíticos de forma atómica en menos de 13 segundos.

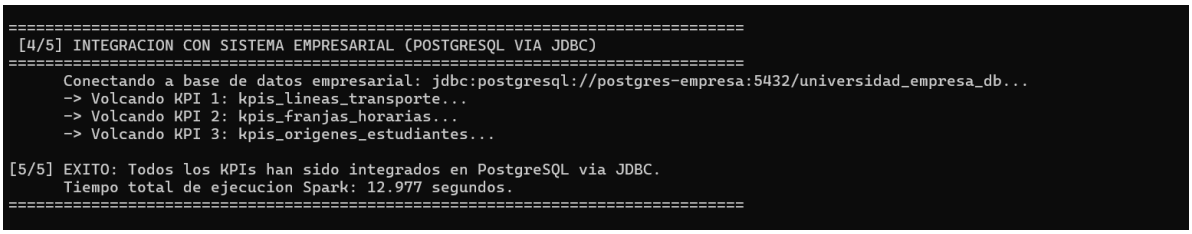


Figura 5: Conexión y volcado exitoso de los DataFrames a PostgreSQL empresarial mediante JDBC en 12,977 segundos.

Adicionalmente, se desarrolló un visor web (`dashboard_movilidad.html`) donde el vicerrectorado puede visualizar los gráficos generados a partir de los datos consolidados por Spark.

6. Análisis e interpretación de los resultados

Los resultados analíticos aportan conclusiones directas para la toma de decisiones:

1. **Efecto embudo matutino:** El **44,88 %** de todos los desplazamientos ocurren en la franja matinal (07:30–09:30 h), sumando 44.879 trayectos. Esto explica la congestión de los aparcamientos y la saturación de las primeras paradas del autobús lanzadera.
2. **Eficiencia del Metro frente al autobús:** El Metro L1 combinado con la lanzadera ofrece el mejor balance entre capacidad (34.993 viajes) y puntualidad (retraso medio de solo 2,11 minutos), mientras que las líneas de autobús en superficie (M-133 y L5) promedian más de 6,8 minutos de demora por retenciones en rotondas de acceso.
3. **Impacto ambiental contrastado:** A pesar de que los vehículos privados individuales solo representan el 6,06 % de los viajes, son responsables de la mayor cuota de

emisiones contaminantes (5.877 kg de CO_2). En contraposición, los estudiantes que eligen transporte público o bicicleta han ahorrado más de **70 toneladas de CO_2** en un solo cuatrimestre.

4. **Rendimiento de Spark SQL:** Procesar 100.000 filas con filtrados, agrupaciones, cálculos de varianza y escrituras remotas JDBC en 14 segundos valida la idoneidad de la computación distribuida en memoria frente a batch tradicionales en disco.

7. Decisiones tomadas y justificación

- **Uso de Apache Spark frente a consultas SQL directas en PostgreSQL:** Ejecutar cálculos analíticos masivos continuos sobre la base transaccional de la universidad causaría bloqueos de tablas y degradación para los usuarios finales. Desacoplar la analítica pesada en Spark protege el sistema de gestión.
- **Elección del módulo Spark SQL frente a RDDs de bajo nivel:** La API de DataFrames y Spark SQL se beneficia de las optimizaciones automáticas del motor Catalyst (reordenación de filtros y ejecución vectorial), generando un código limpio, conciso y estándar en la industria.
- **Integración mediante JDBC en modo Overwrite:** Como los KPIs se recalculan periódicamente en un proceso ETL batch, la escritura con `mode(.ºverwrite")` asegura que el cuadro de mando corporativo consulte siempre una instantánea limpia sin duplicados.
- **Auditoría de ejecución automatizada:** Añadir la tabla `registro_procesamiento_spark` permite que el DBA de la universidad monitorice tiempos de ejecución, volumen de datos procesados y estado de salud de los trabajos Big Data.

8. Conclusiones y trabajo futuro

La realización de este TPI 2 ha permitido comprobar en la práctica el potencial del paradigma Big Data. Se ha experimentado cómo una herramienta como Apache Spark simplifica radicalmente el análisis a gran escala cuando se combina con la familiaridad de la sintaxis SQL.

La conexión entre el TPI 1 (ubicación geográfica y bases espaciales con PostGIS) y el TPI 2 (flujos de movilidad y analítica distribuida con Spark SQL) proporciona una visión integral de cómo las tecnologías avanzadas de bases de datos resuelven facetas complementarias de un problema de ingeniería del mundo real.

Líneas de trabajo futuro:

- **Streaming en Tiempo Real con Spark Structured Streaming:** Sustituir el procesamiento por lotes (batch) por la ingesta continua de eventos en tiempo real mediante Apache Kafka, permitiendo alertas inmediatas de autobuses llenos.
- **Machine Learning predictivo con Spark MLlib:** Entrenar modelos de regresión para predecir retrasos de transporte en función de la meteorología (lluvia en Sevilla) y el calendario académico (semanas de exámenes).

9. Declaración de uso de inteligencia artificial

Marque una opción y complete la información correspondiente:

☐ No he utilizado herramientas de inteligencia artificial en la realización de este trabajo.

■ **Sí he utilizado herramientas de inteligencia artificial como apoyo en este trabajo.**

Herramienta(s) utilizada(s):
Google Gemini / Antigravity.

Uso concreto realizado:
Se utilizó como asistente técnico para estructurar los contenedores en Docker Compose, resolver dudas de sintaxis sobre la integración JDBC de PySpark con PostgreSQL, y dar soporte en la maquetación del informe en LaTeX conforme a la plantilla oficial de la universidad.

Revisión y validación personal:
El estudiante ha ejecutado, probado y analizado personalmente el pipeline en la consola de Docker local. Se han verificado las consultas Spark SQL, se han contrastado los resultados obtenidos en las tablas de PostgreSQL y se ha redactado la memoria de manera reflexiva para asegurar el pleno dominio de los conceptos en la defensa oral.

10. Referencias

Referencias

[1] Apache Software Foundation. *Apache Spark SQL, DataFrames and Datasets Guide (v3.5.1)*. 2024.
https://spark.apache.org/docs/3.5.1/sql-programming-guide.html

[2] Apache Software Foundation. *Spark SQL JDBC To Other Databases*. 2024.
https://spark.apache.org/docs/3.5.1/sql-data-sources-jdbc.html

[3] Armbrust, M., Xin, R. S., Lian, C., et al. *Spark SQL: Relational Data Processing in Spark*. Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, pp. 1383–1394, 2015.

[4] PostgreSQL Global Development Group. *PostgreSQL JDBC Driver Documentation (Version 42.7)*. 2024.
https://jdbc.postgresql.org/documentation/

[5] Dr. Anier Soria Lorente. *Orientaciones para el alumnado: Trabajos Prácticos Individuales (TPI) - Ampliación de Bases de Datos*. Universidad Loyola Andalucía, Curso 2026/2027.